

Roteiro revisado — próximos 3 meses

Angular + TypeScript no front-end, C# + ASP.NET Core + Oracle no back-end

Este roteiro substitui o plano original (React + Spring Boot). Duas decisões de stack mudaram no meio do caminho: o front-end passou de React para Angular, e o back-end de Spring Boot para ASP.NET Core (C#). O plano abaixo foi recalibrado considerando essas trocas — e também considerando o ritmo real observado no primeiro mês de projeto.

Onde você está agora (base para este roteiro)

Em aproximadamente 27 dias de projeto, você entregou: 16 telas completas em HTML/CSS (zero estilo inline, organizadas em subpastas por módulo), uma treeview de navegação funcional em JavaScript puro, e — mais importante para este roteiro — JavaScript orientado a objetos genuinamente sólido: classes com construtor, encapsulamento, callbacks para desacoplamento entre componentes (o padrão que você usou na classe de paginação é, na prática, o mesmo princípio por trás de Inputs/Outputs no Angular). Isso é bem além do que o plano original previa para este estágio.

Por isso, este roteiro assume uma curva de aprendizado mais rápida que a média para TypeScript e para os conceitos de Angular que espelham o que você já pratica (componentes, serviços, injeção de dependência via callback/construtor). Onde o roteiro NÃO acelera é em pontos que historicamente travam qualquer pessoa, independente do ritmo: o sistema de módulos/injeção de dependência do Angular na primeira exposição, e principalmente a integração entre Entity Framework Core e Oracle, que tem fricção conhecida de mercado (driver, connection string, sequences vs. identity). Esses pontos têm folga extra reservada.

Semanas	Foco principal	Entregável no fim do bloco
1-2	Fechar pontas do JS vanilla (editar/remover, localStorage) + TypeScript do zero	CRUD completo em memória (Produtos) com persistência local; primeiro script .ts compilado e rodando
3-5	Angular: fundamentos (componentes, templates, routing, Inputs/Outputs)	Login, Dashboard e Sidebar/TreeView convertidos para componentes Angular reais (SPA)
6-8	Angular: serviços, injeção de dependência, RxJS básico, Reactive Forms	Produtos e Clientes 100% em Angular (listagem + cadastro), dados ainda mockados em serviço
9-10	C# do zero (sintaxe, OOP, LINQ)	Pequenos programas de console em C#, aplicando os mesmos conceitos de OOP do JS
11-12	ASP.NET Core Web API + Entity Framework Core + conexão real com Oracle	Fatia fina 100% funcional: Produtos conectado de ponta a ponta (Angular → API → Oracle)

12 semanas a 20h/semana = 240 horas, o equivalente a ~3 meses corridos. A meta final continua a mesma do plano original: não um ERP completo, mas um fluxo fino e totalmente funcional de ponta a ponta, servindo de base sólida para os demais módulos.

Semanas 1-2 — Fechar o JS vanilla + TypeScript do zero

Objetivo: terminar o ciclo de CRUD em memória que você já começou (falta editar e remover), e usar isso como ponte direta para TypeScript — que, no seu caso, é menos "aprender do zero" e mais "aprender a dar nomes formais ao que você já faz".

Semana 1 — Fechar o CRUD em memória (JavaScript puro)

Carga estimada: ~20h

- Implementar remover: usar `.filter()` para excluir um item pelo id, reaproveitando o padrão de delegação de eventos já usado no treeview
- Implementar editar: reaproveitar o mesmo formulário de criar, controlado por uma variável de estado (`idEmEdicao`), como já discutido
- Persistir o array em `localStorage`, para os dados sobreviverem ao recarregar a página
- Aplicar o mesmo padrão (classe de listagem + paginação) em pelo menos mais uma tela (Produtos), validando que a arquitetura realmente se reaproveita

Entregável: CRUD completo (criar/ler/editar/remover) funcionando em pelo menos duas telas, com paginação reaproveitada e dados persistindo entre recarregamentos

Semana 2 — TypeScript: fundamentos

Carga estimada: ~20h

- Instalar TypeScript e entender o papel do compilador (tsc): TS não roda no navegador, ele vira JavaScript antes de rodar
- Tipos básicos (string, number, boolean, array, any) e por que evitar "any"
- Interfaces — modelar Produto, Cliente, Usuário como interfaces, espelhando os objetos que você já usa em JS
- Classes em TypeScript: comparar com as classes que você já escreveu em JS (o que muda é basicamente a tipagem de propriedades e parâmetros, não o conceito)
- Reescrever a classe de paginação e uma das classes de listagem em TypeScript, como exercício de tradução direta

Entregável: A classe de paginação e pelo menos uma listagem reescritas em TypeScript, compilando sem erros e funcionando de forma idêntica à versão em JavaScript

Semanas 3-5 — Angular: fundamentos

Objetivo: sair do zero em Angular e chegar a um app real rodando, com pelo menos três telas convertidas — usando o Angular CLI para gerar a estrutura correta desde o início, em vez de tentar montar isso manualmente.

Semana 3 — Angular CLI, estrutura de projeto e primeiro componente

Carga estimada: ~20h

- Instalar Angular CLI e criar o projeto (ng new) — decidir agora se reaproveita a pasta front/ (refazendo do zero) ou cria uma pasta nova
- Entender a anatomia de um componente Angular: arquivo .ts (lógica), .html (template), .css (estilo) — o CSS que você já escreveu em componentes.css vira referência direta para os estilos de cada componente
- Data binding: interpolação ({{ }}), property binding ([]), event binding (())
- Diretivas estruturais (*ngFor, *ngIf) — o equivalente direto do seu .map() e dos seus if/else de mostrar-ou-esconder elemento
- Converter o Login para um componente Angular

Entregável: Login funcionando como componente Angular, rodando via ng serve

Semana 4 — Componentização da Sidebar/TreeView

Carga estimada: ~20h

- Extrair a sidebar (hoje copiada e colada em 16 arquivos HTML) para UM componente Angular único, reaproveitado em todas as páginas — o problema que você mesmo identificou ("a mesma sidebar em vários arquivos") se resolve aqui de vez
- @Input() e @Output(): passar dados para o componente (qual item está ativo) e emitir eventos para o componente pai (clique em um item do menu)
- Reescrever a lógica de expandir/recolher (nextElementSibling + classList.toggle) usando o jeito Angular de esconder/mostrar elementos, controlado por uma propriedade do componente, não por manipulação direta do DOM
- Converter o Dashboard para consumir esse componente de sidebar

Entregável: Um único componente de sidebar, usado por Login e Dashboard, sem nenhuma duplicação de HTML — resolvendo de vez o problema de manutenção que a versão em HTML puro tinha

Semana 5 — Angular Router

Carga estimada: ~20h

- Configurar rotas para as páginas já existentes (login, dashboard, produtos, clientes...)
- Trocar os links de navegação (href para arquivo .html) por routerLink — a navegação deixa de recarregar a página inteira (SPA de verdade, o problema que você identificou de "perder o estado ao trocar de página" desaparece aqui)
- Marcar visualmente o item ativo da sidebar usando routerLinkActive, em vez de controlar isso manualmente por classe em cada arquivo

Entregável: Navegação entre pelo menos 4 telas (Login, Dashboard, Produtos, Clientes) sem recarregar a página, com a sidebar já refletindo automaticamente a rota atual

■ **Ponto de atenção:** o sistema de módulos e injeção de dependência do Angular tem uma curva de estranhamento na primeira exposição, mesmo para quem já programa bem. Se a Semana 5 render menos que

o esperado, é normal — não é sinal de que você está indo mal.

Semanas 6-8 — Angular: serviços, RxJS e formulários

Objetivo: dar "vida" aos dados. Até aqui, os componentes têm dados fixos escritos neles mesmos. Agora os dados passam a vir de serviços — a peça que, mais para frente, vai simplesmente trocar de "array fixo" para "chamada real à API", sem precisar reescrever os componentes.

Semana 6 — Services e injeção de dependência

Carga estimada: ~20h

- Criar um Service (ex: ProdutoService) que guarda o array de produtos — a mesma ideia do seu array solto em produtos.js, só que agora "injetável" em qualquer componente
- Entender injeção de dependência: por que o Angular "entrega" instâncias de serviço para os componentes, em vez de você criar elas manualmente com o "new"
- Mover a lógica de criar/editar/remover/paginar para dentro do serviço, deixando os componentes só responsáveis por mostrar dados na tela

Entregável: ProdutoService e ClienteService funcionando, com os componentes de listagem consumindo dados através deles, não mais de arrays soltos no próprio componente

Semana 7 — RxJS básico (Observables)

Carga estimada: ~20h

- Entender por que Angular usa Observables em vez de retornar dados diretamente — a ideia central de "dado que chega ao longo do tempo", que fica óbvia quando, no mês 3, os dados passarem a vir de uma API
- Operadores básicos: map, filter (do RxJS) — reaproveitando a intuição que você já tem do .map()/filter() do JavaScript puro, só que aplicados a um fluxo de dados, não a um array parado
- subscribe() e por que "nada acontece" em um Observable até alguém se inscrever nele

Entregável: Os serviços de Produtos e Clientes expõem Observables em vez de arrays diretos, com os componentes usando o "pipe async" do Angular para exibir os dados

Semana 8 — Reactive Forms

Carga estimada: ~20h

- Reactive Forms do Angular — construir o formulário de Cadastro de Produto usando FormGroup e FormControl, com validadores (required, min, pattern)
- Comparar diretamente com sua validação manual em JS (if vazio, mostrar erro) — o Angular formaliza exatamente esse padrão que você já pratica, com menos código repetido
- Cálculo de margem de lucro em tempo real, reescrito com valueChanges do formulário reativo, no lugar do addEventListener('input', ...) manual

Entregável: Cadastro de Produto 100% funcional em Angular: validação, cálculo em tempo real, criação e edição — usando o serviço criado nas semanas anteriores

Semanas 9-10 — C# do zero

Objetivo: aprender a sintaxe e as convenções de C#, apoiando-se fortemente no que você já sabe de orientação a objetos (classes, construtores, encapsulamento, herança) — esses conceitos não mudam, só a sintaxe muda.

Semana 9 — Sintaxe e OOP em C#

Carga estimada: ~20h

- Tipos, variáveis, estruturas de controle — sintaxe muda pouco em relação ao que você já conhece de JS/TypeScript
- Classes em C#: propriedades (get/set), construtores, encapsulamento — comparando diretamente com as classes que você já escreveu (a classe de paginação seria um bom exercício de tradução para C#, só como estudo, sem uso real ainda)
- Herança e interfaces em C# — muito próximo do que você já perguntou sobre "override" em JS, só que aqui é uma exigência formal da linguagem, não opcional

Entregável: Pequenos programas de console em C# aplicando classes, herança e interfaces — sem interface gráfica ainda, só para fixar a linguagem

Semana 10 — LINQ e coleções

Carga estimada: ~20h

- LINQ (Language Integrated Query) — o equivalente direto do .map()/.filter() que você já domina em JavaScript, só que embutido na linguagem C#
- Coleções (List, Dictionary) e diferenças em relação a arrays de JS
- async/await em C# — parecido conceitualmente com Promises em JS, mas com sintaxe própria; importante porque toda chamada ao banco de dados no ASP.NET Core usa isso

Entregável: Exercícios de LINQ replicando as mesmas operações que você já fez em JS (filtrar produtos ativos, buscar por nome, etc.), agora em C#

Semanas 11-12 — ASP.NET Core + Entity Framework Core + Oracle

Objetivo final do trimestre: fechar o ciclo completo. Esta é a parte do roteiro com mais risco de atraso — não pela sua capacidade, mas porque a combinação Entity Framework Core + Oracle tem particularidades conhecidas que não aparecem com bancos mais comuns como SQL Server ou PostgreSQL.

Semana 11 — ASP.NET Core Web API: fundamentos

Carga estimada: ~20h

- Criar o projeto Web API (dotnet new webapi) e entender a estrutura gerada
- Controllers e rotas — o equivalente direto dos endpoints que existiriam no Spring Boot do plano original, só que em C#
- Injeção de dependência nativa do ASP.NET Core — reaproveitando a mesma intuição que você constrói agora com o Angular (os dois frameworks usam o mesmo princípio)
- Criar um endpoint GET /produtos retornando uma lista fixa em memória (ainda sem banco), só para validar a comunicação Angular → API

Entregável: Angular chamando um endpoint real via HttpClient, substituindo o serviço mockado por uma chamada de rede de verdade — mesmo que os dados ainda não venham do Oracle

Semana 12 — Entity Framework Core + Oracle

Carga estimada: ~20h

- Instalar o provedor Oracle para Entity Framework Core (Oracle.EntityFrameworkCore)
- Configurar a connection string e mapear a entidade Produto para uma tabela real
- Migrations do EF Core — gerar a tabela no banco a partir da classe C#
- Implementar o CRUD completo do endpoint de Produtos usando o banco de verdade
- Testar o fluxo completo: cadastrar um produto pelo Angular, dar F5, e confirmar que ele continua lá — a prova final de que a persistência funciona de ponta a ponta

Entregável: Meta do trimestre atingida: Login + Dashboard + Produtos (listagem e cadastro) funcionando de ponta a ponta — Angular → ASP.NET Core → Oracle

■ **Ponto de atenção real:** Oracle tem particularidades que Postgres/SQL Server não têm (uso de sequences em vez de colunas identity/auto-incremento, sensibilidade a maiúsculas/minúsculas em nomes de tabela, tipos de dados um pouco diferentes). Se esta semana render menos, considere isso normal, e não hesite em reservar dias extras aqui — é melhor atrasar esta parte do que pular etapas de configuração do banco.

O que fica para depois desses 3 meses

Para manter o escopo realista, os itens abaixo são conscientemente adiados — isso não é atraso, é o plano funcionando como deveria.

- Autenticação real (login com token JWT) — o Login em Angular, ao fim deste roteiro, ainda é só validação de formulário, sem autenticação de verdade contra o back-end
- As demais telas (Pedidos, Usuários/Permissões, Config) convertidas para Angular e conectadas ao back-end — o roteiro foca em Produtos como fatia vertical completa
- Módulos ainda nem desenhados: Fornecedores, Compras, Financeiro
- Testes automatizados (unitários ou de integração), tanto no Angular quanto no ASP.NET Core
- Deploy em ambiente real / produção
- Refinamento de UX que dependa de dados reais (ex: paginação server-side, em vez de paginar um array já carregado inteiro no front-end)

Recomendações gerais para os próximos 3 meses

- Se o ritmo das Semanas 1-2 (JS/TypeScript) continuar tão rápido quanto o primeiro mês, não acelere o calendário — use o tempo excedente para consolidar Angular com mais profundidade (documentação oficial, não só o suficiente para copiar o padrão)
- Ao chegar nas Semanas 3-5, resista à tentação de recriar todas as 16 telas de uma vez em Angular — o roteiro propõe só 3-4 telas propositalmente, para não repetir o padrão de "muito HTML, pouca lógica" que já foi identificado e evitado antes
- A escolha de reescrever a classe de paginação em TypeScript (Semana 2) e depois como estudo em C# (Semana 9) é proposital — comparar a mesma lógica em três linguagens diferentes é um dos jeitos mais eficazes de fixar que o conceito importa mais que a sintaxe
- Reserve as Semanas 11-12 com expectativa flexível — é a parte do roteiro com maior chance real de imprevisto técnico (Oracle + EF Core), não por falta de capacidade, mas por ser território genuinamente mais cheio de detalhes de configuração
- Ao final da Semana 12, atualize o README do repositório e este roteiro antes de planejar o próximo trimestre — os últimos dois roteiros ficaram desatualizados por algumas semanas depois de escritos